



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria dell'Automazione e Robotica

Thesis of Control Lab

Executing Efficient Lane Keeping with Duckietown's Mobile Differential Robot

Professor Raffaele Iervolino

Candidates

- D'Angelo Antonio *P38000115*
- Maffeo Marco *P38000211*
- Molaro Vincenzo *P38000217*
- Riccardi Stefano *P38000210*
- Torella Gaetano *P38000200*
- Tuccillo Domenico *P38000215*
- Vitone Carlo *P38000227*
- Vodola Benito *P38000216*

Abstract

The Duckiebot is a differential drive robot designed and developed by Massachusetts Institute of Technology in 2016. The Duckietown platform has been used extensively for research on mobile robotics and physically embodied AI systems. The key idea of our thesis project is to control the duckiebot exploiting previous knowledge in the control theory adapting them with real sensor and actuator.

Contents

Abstract	i
1 Duckietown of DDMR	1
1.1 Construction and Setup of the Duckiebot	1
1.2 Hardware	8
2 Simulations	10
2.1 Cad Model on Simulink	10
2.1.1 Wheel rotation setting	13
2.1.2 Road Contact	16
2.2 Control Model	17
2.2.1 Motor Driver Subsystem	19
2.2.2 DC Motor Subsystem	19
2.3 Dynamic Model of the DDMR	21
3 Ros toolbox	25
3.1 Integrating ROS Toolbox for Remote Communication in Duckiebot Control Design	25
3.1.1 Modelling of the ROS command	27

4	Inner Loop	29
4.1	Reconstruction of the velocity from the encoder data	29
4.2	Velocity Control and Dynamic Simulation	30
4.3	Dynamic Modeling and Control Adaptation	31
5	Outer Loop	33
5.1	Colour Detection	33
5.2	Line Detection	34
5.3	Line Keeping	37
5.4	Control Merge	42
6	Conclusion	43

Chapter 1

Duckietown of DDMMR

1.1 Construction and Setup of the Duckiebot

The first step to do is to build the robot connecting to the chassis the actuators, the micro-controller board, the camera and the power supply. The actuators are two 5 Volt DC motors with Hall effect encoders connected to the wheels. The encoders count 137 ticks/revolution (2.6deg resolution).

external world where the duckiebot works.

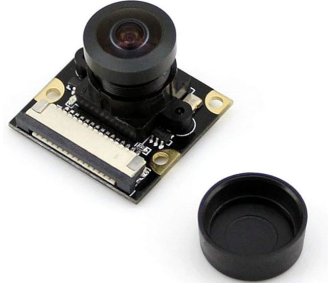


Figure 1.3: camera sensor

Then, the power supply is a power bank of dimensions: 62 x 100 x 24.5 mm; weight: 189g (fully charged) rated capacity: 10 Ah at 3.7V (37Wh). It can be charged via Micro USB 5V at up to 2A and has two USB type A outputs.



Figure 1.4: duckiebattery

Fixing these components to the chassis by using screws and bolts and connecting the hardware components by using the jumper wires

and FPC cable, the duckiebot is now completely built.

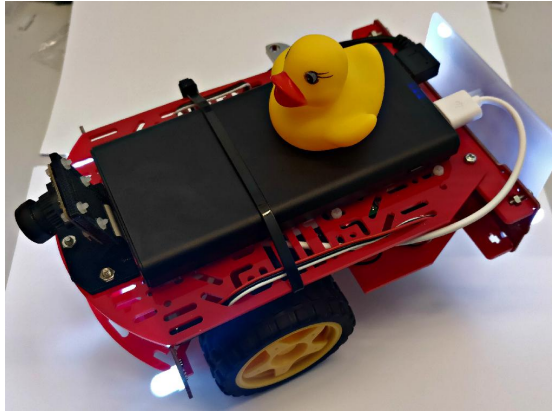


Figure 1.5: duckiebot

Now it is possible to execute the setup of the robot and install all the software to run Duckietown development environment on laptop. To do that, it is necessary to have a Micro SD card of 32 gigabyte. Duckietown supports as operating system Ubuntu 20.04 and MacOSX, therefore is not possible to work with a Windows machine. Moreover, the duckiebot uses the Robot Operating System to communicate with the actuators and the sensors. By following the steps that are reported on Duckietown website, it is possible to download all the necessary dependencies and the Duckietown dashboard.

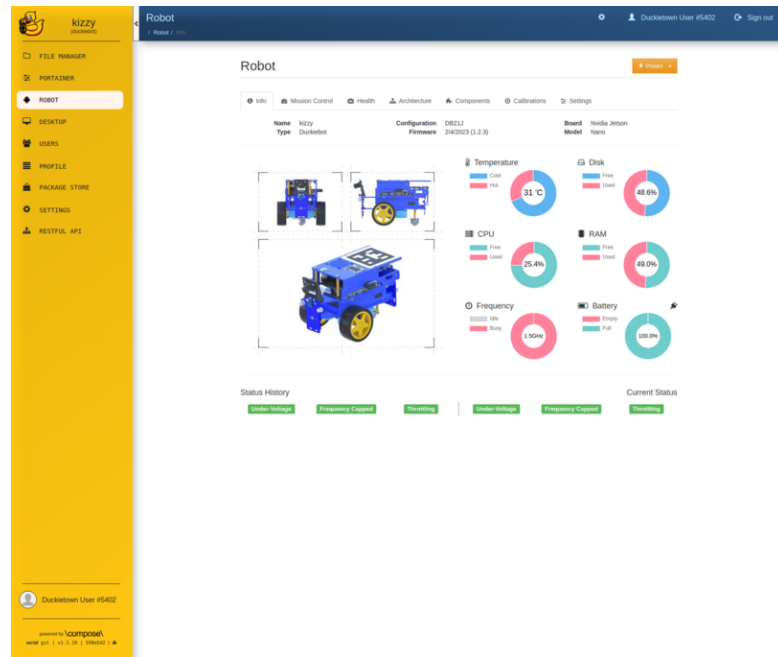


Figure 1.6: dashboard

The Duckietown Dashboard has different sections that allow the user to check all the information about the robot. Indeed, the dashboard has a section for the camera that reproduces the image captured by the camera; a section to detect the angular velocity of the wheels and a section to study all the ROS topics.

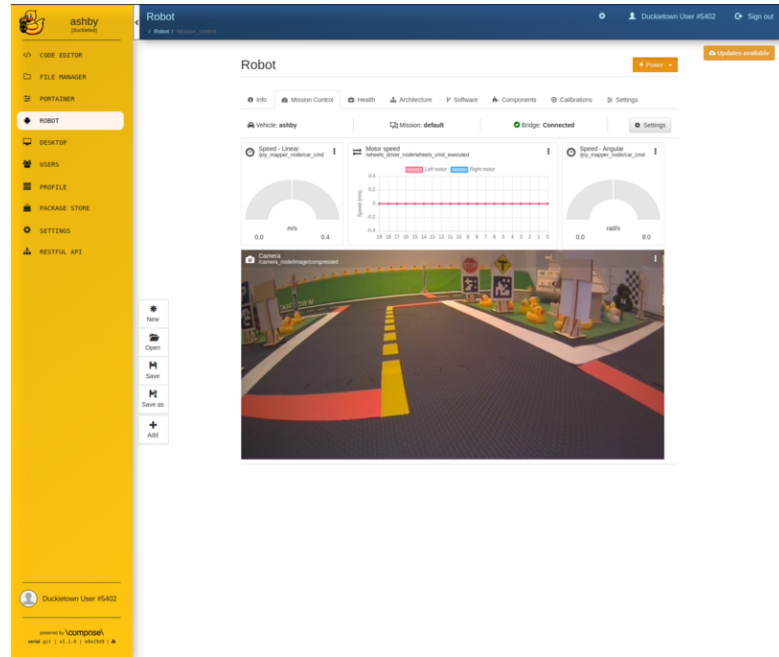
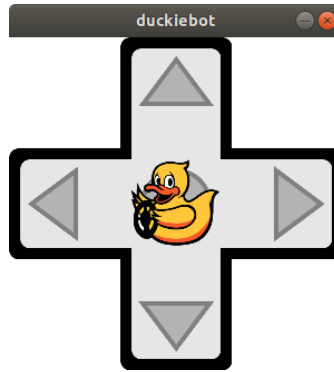


Figure 1.7: dashboard camera

To use the Duckietown Dashboard is necessary to install the Duckietown Shell and create an account. After this step is mandatory to insert the SD card to execute the flashig operation. This operation can be done by using a GUI or by command line. In this step we have to choose the name of the robot and set the Networks that we want our robot connect to.

Figure 1.8: Flashing sd card

From the command line is it possible to connect the duckiebot to our laptop. It is mandatory that the laptop and the robot are connected to the same network. Once the robot has been connected to the laptop, after confirming the first boot, you can finally acces to the Dashboard. In the end it is possible to make the robot move by using a keybord control provided by the Duckietown shell from the command line in Ubuntu. After startup, an interface window will be opened. By clicking the arrow keys the duckiebot will move towards the desired direction.

Figure 1.9: `keyboardgui`

1.2 Hardware

The DuckieBot is equipped with a Raspberry 3B+ board which is connected to a camera with 160-FOV fisheye lens and an Adafruit PWM servo HAT that controls the LEDs and motors. The motors are two 5V DC motor equipped with encoder with a resolution of 2.6 degrees (135 ticks/revolution) based on two Hall effect sensor for each motor. During the first tests, we tried to derive the information coming from the motor encoders and the camera through the tools offered by the "Support Package for Raspberry Pi Hardware" and the same thing was done to send commands to the engines. We configured the Raspberry Pi using the toolbox provided by MATLAB and Simulink. We used Encoder, GPIO Read, and Video Capture blocks to obtain information from the machine. However, we discovered that the Raspberry Pi communicates with the HAT using a communication protocol called I2C, which uses specific registers

and an information encoding standard to send and receive information to the encoders and motors. We searched for documentation on this and focused our attention on Duckietown’s GitHub repository, where we learned that communication between various components occurs through ROS nodes encoded in the Python language. After days of reading thousands of lines of code and hundreds of files, we didn’t find anything that could help us. For these reason we decide to focus our attention on ROS.

Chapter 2

Simulations

2.1 Cad Model on Simulink

In our initial stages, we embarked on a quest to find a suitable model on Onshape that could effectively emulate the dynamics of our duckie. Initially contemplating a straightforward drone model with two active wheels and one passive wheel, our search led us to the DuckiebotV9 model, a discovery that exceeded our expectations. The DuckiebotV9 model comprises seven main components, with each part housing around a hundred smaller subparts for the assembly process. To simplify, these components include:

- SBC (Single Board Computer): Consisting of Raspberry and Duckiehat
- Wires

- Battery
- Front Assembly
- Back Assembly: Featuring the blackpattern plate
- Top Assembly: Encompassing the cam hold
- Bottom Assembly: Comprising left motor, right motor, and Hudson bearing

Despite the model’s impressive level of detail, with over 650 parts, it posed challenges for seamless integration into the Simulink environment. Consequently, we undertook specific modifications:

1. Elimination of Redundancies: Extraneous elements such as screws, pins, bolts, and cables were removed to streamline the model.
2. Material Assignment: Materials such as PVC and aluminum were assigned to the model for enhanced realism in the simulation.
3. Assembly Streamlining: We created a consolidated assembly of all part assemblies, facilitating efficient fastener coupling.

Following the successful import into Simulink, additional adjustments were made to ensure the model’s adaptability for inspection. Initially, on Onshape, all parts were rigidly coupled, maintaining

this cohesion on Simulink. Consequently, each of the seven assembly blocks in Simulink comprises individual components with consistent reference frames. In summary, we now have seven blocks, each representing an assembly, with corresponding components and reference frames aligned to ensure a coherent and effective simulation process.

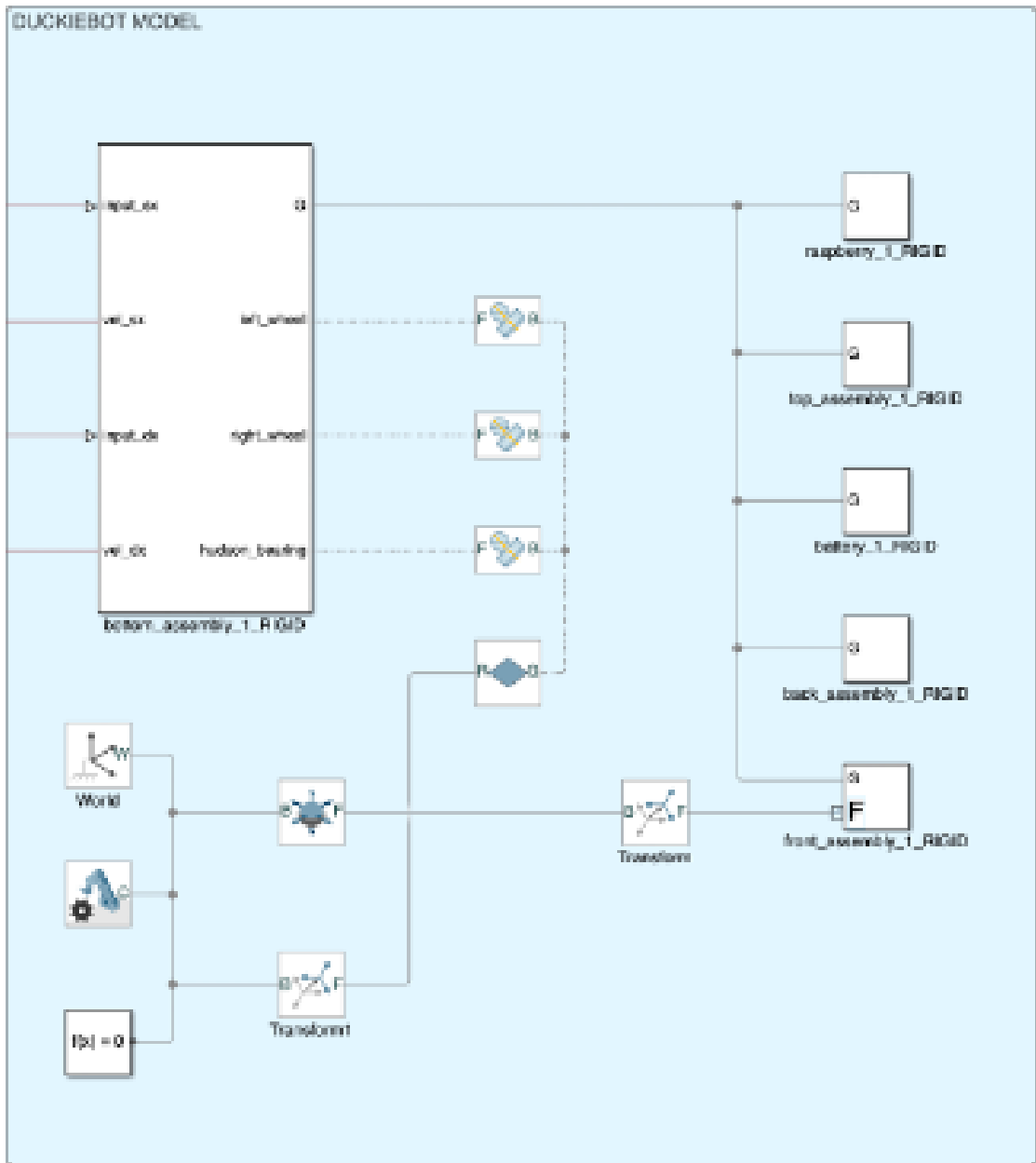


Figure 2.1: Duckiebot Model

2.1.1 Wheel rotation setting

We can observe that our primary focus has been within the bottom assembly. From the original bottom assembly, we selectively extracted only the parts that were relevant to us, specifically those pertaining to the three wheels. Concerning the mobile wheels, we organized them into two subsystems named "left motor 1 RIGID" and "right motor 1 RIGID." As for the passive wheel, we set it aside for the time being, as we'll address it later in the discussion.

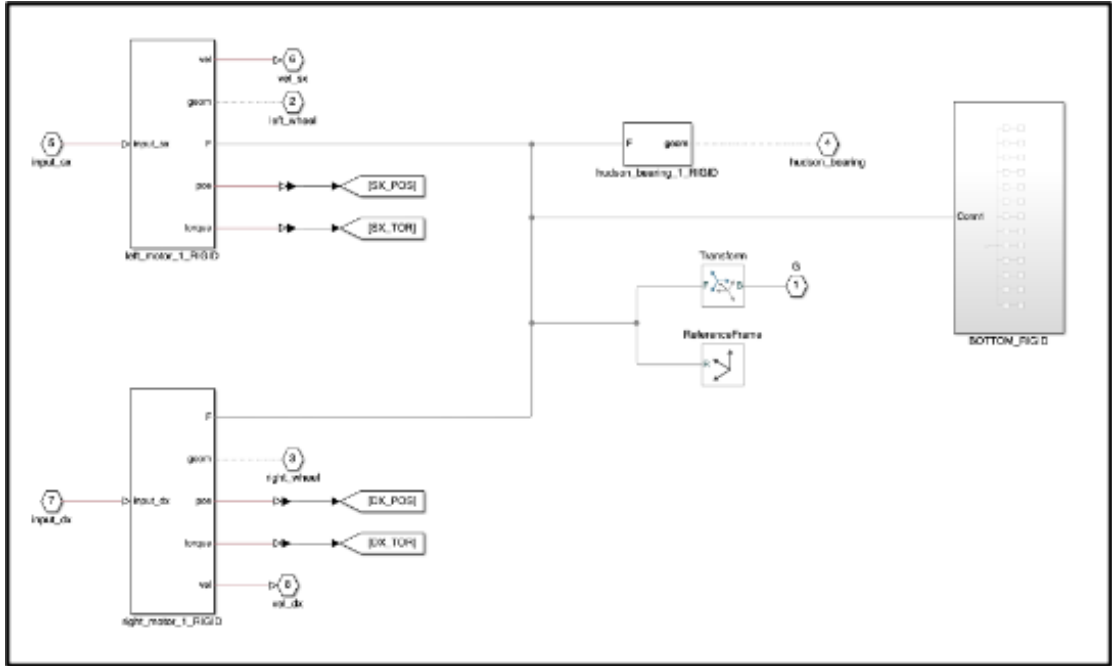


Figure 2.2: Bottom Assembly

We will conduct an analysis of only one of the two wheels since the process for composing the other will be identical. I have chosen the left wheel for this demonstration. Below is the Simulink diagram representing the left wheel [Figure 3].

As evident from the diagram, all components unrelated to rota-

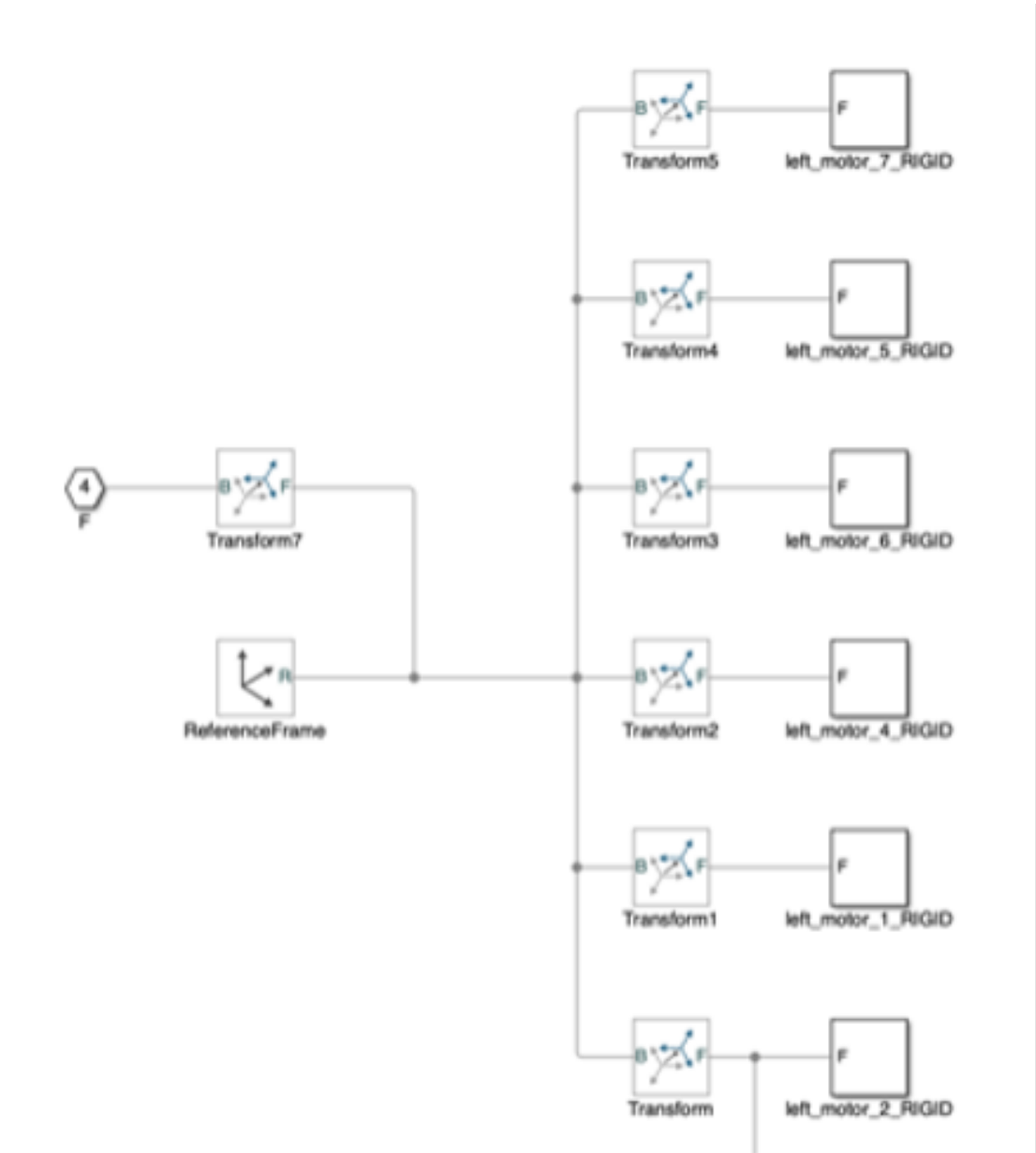


Figure 2.3: Left wheel

tion are left unchanged, and our focus is directed solely towards "left motor 2 RIGID" (the blue motor on the left), "left motor 3 RIGID" (the gray pin situated between the motor and the wheel), and "left wheel 1 RIGID" (the left wheel).

The motor's rotation is simulated by the gray pin through a rev-

olute joint, activated by an input torque. The joint yields position, velocity, and torque as output parameters. This joint is strategically positioned between the motor and the gray pin, allowing the pin to rotate with respect to a reference frame that we defined. To achieve this, we customized the rotation and translation matrix values within the Transform8 block with the following parameters:

Rotation: $[0 \ 1 \ 0]$ at 90 degrees, Translation: $[13.9 \ 11 \ 0]$

These specific values in the rotation and translation matrices enable precise simulation of the motor's rotation, capturing the dynamic interaction between the motor, pin, and wheel. This methodology ensures an accurate representation of the left wheel's behavior within our Simulink model [Figure 4].

Note: The values were chosen gradually based on the displacement of the reference axis.

In summary, the left wheel will no longer be fixed with respect to the entire bottom assembly but will instead be rigidly attached to the left motor. Consequently, when the motor rotates, the wheel will rotate accordingly. That's all for the rotation; now let's delve into how we've managed to make the vehicle move on a plane.

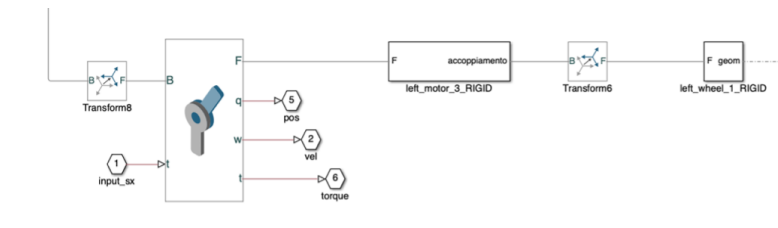


Figure 2.4: Left whell Rotation

2.1.2 Road Contact

Everything related to this aspect is depicted in Simulink through the dashed connections. We've established a plane, measuring 1m x 1m, upon which the vehicle is positioned, utilizing the Infinite Plane block.

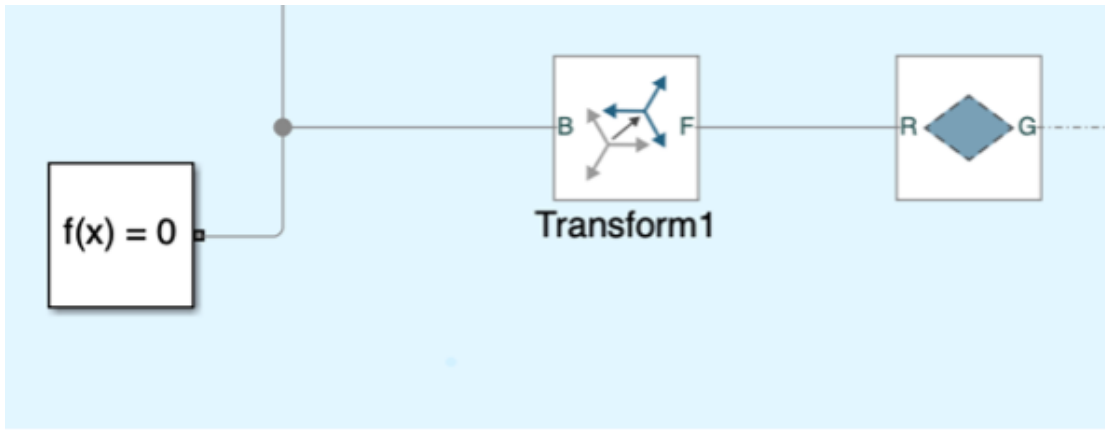


Figure 2.5: Infinite plane block

On this plane, we've placed the three elements for movement: the tire of the right wheel, the tire of the left wheel, and the sphere of the passive wheel. To achieve this, we utilized the Convex Hull function within the Solid block for the right and left wheel tires [Figure 6].

The same approach was employed for the sphere of the passive wheel, utilizing the Convex Hull function within the Solid block [Figure 7].

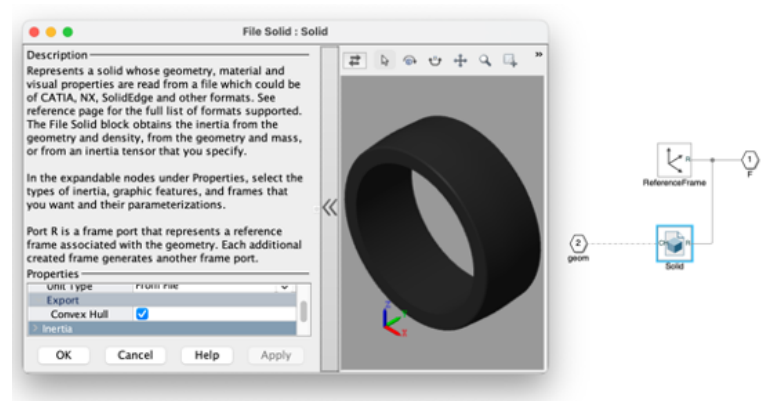


Figure 2.6: Left wheel Rubber

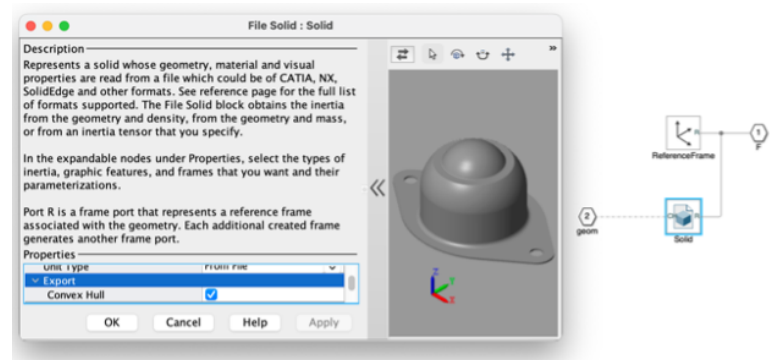


Figure 2.7: Passive wheel

At this point, the last step is to connect the aforementioned elements to the plane. To achieve this, we employ a contact constraint, and the linkage is established through this method. At the other end, we naturally have the plane that we created earlier [Figure 8].

2.2 Control Model

An early step in controlling our Duckiebot was to implement a controller in a model created in SimScape. In this model, we chose

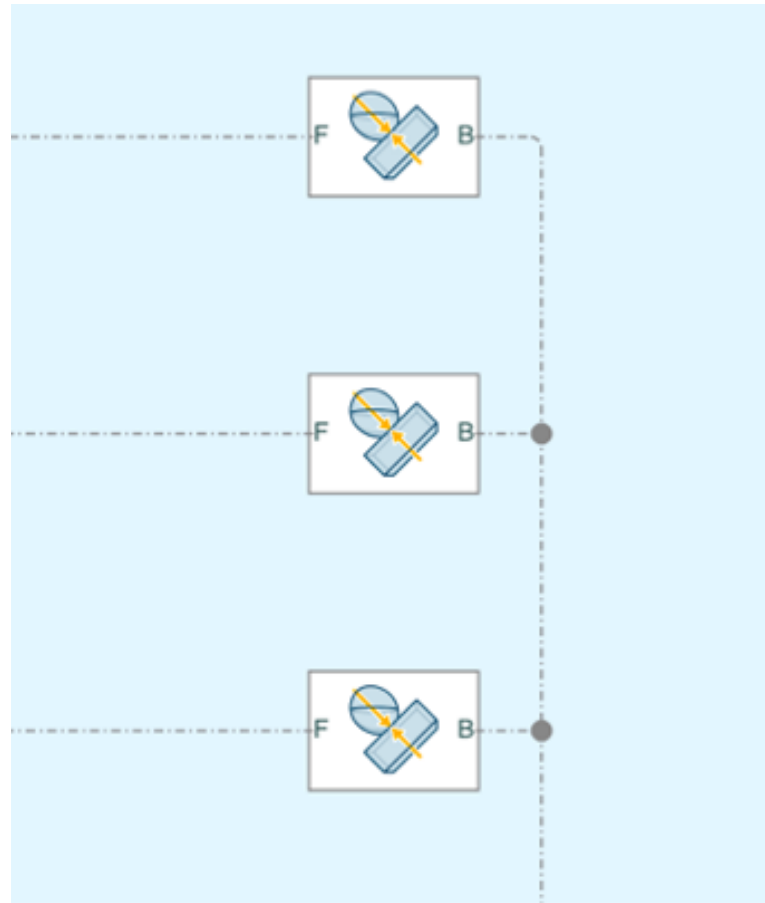


Figure 2.8: Contact constrains

a DC motor called `Maxon_Amax_19_352922` due to complications regarding the specification of the real motor.

The block `"motor driver SX"` is the one that controls the DC motor. It uses the signal given by PID and employs a PWM and an H-bridge to generate the voltage for the motor. The feedback of the system is the Joint velocity [rpm], filtered by a low-pass filter.

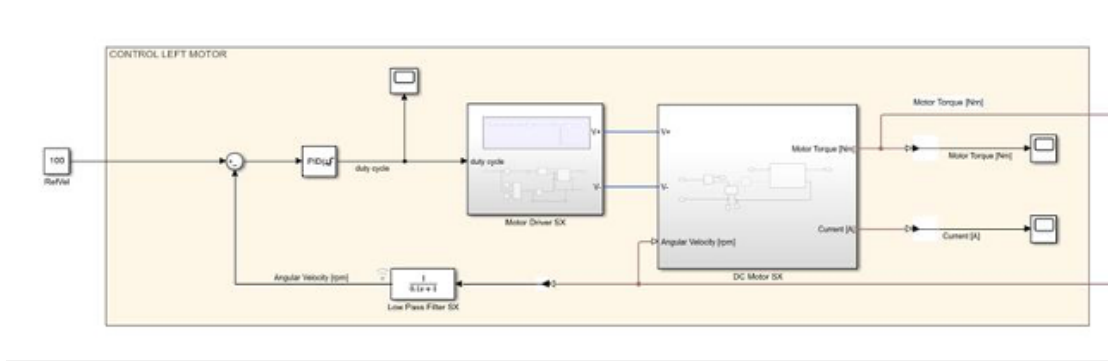


Figure 2.9: Simulink model

2.2.1 Motor Driver Subsystem

The input to this subsystem is the duty cycle, generated by a PID controller.

Controlled PWM Voltage Block: Generates a PWM signal based on the value of the input duty cycle. The PWM signal is used to control the power supplied to the motor.

H-Bridge Block: An H-Bridge is an electronic circuit that allows a DC motor to be powered in both directions. PWM modulation is commonly used in motor controls because it allows for efficient control of motor speed without dissipating much heat [Figure 10].

2.2.2 DC Motor Subsystem

In the DC motor subsystem, the input includes a power supply with positive $V+$ and negative $V-$ terminals. This serves as the power input for the motor. For this simulation, we chose a DC motor: "Maxon:Amax_19_352922".

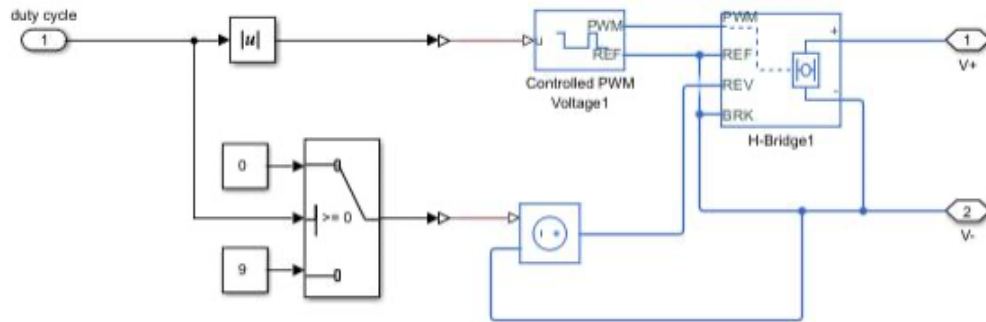


Figure 2.10: Motor Driver Subsystem

There is also a "Rotational Simscape Intf" block, which is a rotational motion interface in the Simscape environment, part of MATLAB/Simulink used for simulating physical systems. The output of this subsystem is the motor torque, which serves as the input for the rotational joint [figure 11].

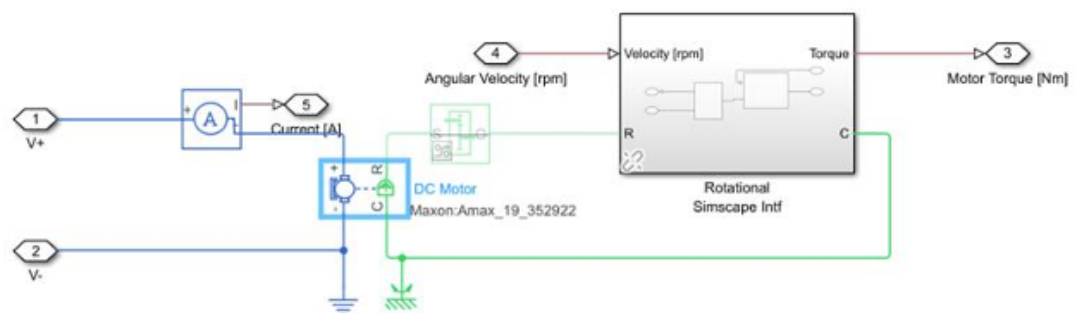


Figure 2.11: DC Motor Subsystem

2.3 Dynamic Model of the DDMR

A dynamic model of a differential mobile robot describes the behavior and dynamics of the robot while it moves. A differential mobile robot consists of two wheels mounted on separate axles that can move at different speeds, enabling the robot to perform complex movements such as spinning in place and moving in various directions.

The dynamic model takes into account various factors such as the kinematics and dynamics of the robot's movement, wheel geometry, friction, robot mass, and so on. This model allows predicting how the robot will move in response to specific inputs, such as wheel speeds or applied forces. In our case, the dynamic model has been useful in order to calibrate the proportional, integral, and derivative gains of the PID controller.

In this paragraph, we will solely present the relationships of the dynamic model without referencing the mathematical steps used to derive it, employing equivalently either the Newton-Euler method or Lagrange's method.

The utilized dynamic model is a nonlinear model described exclusively by the two following equations:

$$\left[\frac{R(Md^2 + J)}{4L^2} + \frac{MR}{4} \right] \ddot{\phi}_R + \left[-\frac{R(Md^2 + J)}{4L^2} + \frac{MR}{4} \right] \ddot{\phi}_L \quad (2.1)$$

$$+ \left[\frac{MdR^2}{4L^2} \right] \ddot{\phi}_L^2 + \left[\frac{MdR^2}{4L^2} \right] \dot{\phi}_R \dot{\phi}_L = \frac{1}{R} \tau_R \quad (2.2)$$

$$\left[\frac{R(Md^2 + J)}{4L^2} + \frac{MR}{4} \right] \ddot{\phi}_L + \left[-\frac{R(Md^2 + J)}{4L^2} + \frac{MR}{4} \right] \ddot{\phi}_R \quad (2.3)$$

$$- \left[\frac{MdR^2}{4L^2} \right] \ddot{\phi}_R^2 + \left[\frac{MdR^2}{4L^2} \right] \dot{\phi}_R \dot{\phi}_L = \frac{1}{R} \tau_L \quad (2.4)$$

In this case, the inputs to our dynamic model are represented by the torque couples τ_R and τ_L , developed respectively by the motors on the right and left wheels. Meanwhile, the outputs are represented by the angular position and angular velocity of the right wheel, ϕ_R and $\dot{\phi}_R$, and by the angular position and angular velocity of the left wheel, ϕ_L and $\dot{\phi}_L$.

Within the model, there are certain parameters, where: M stands for the mass of the differential mobile robot, J represents the moment of inertia relative to the robot, R denotes the radius of the wheel, L signifies half the distance between the two wheels, and d is the distance from the axis between the two wheels to the center of mass. The aspect of interest was determining these parameters. The simplest parameters to determine were the mass M , the semi-axis

length L , and the wheel radius R , as they were calculated through direct measurements using a scale and a ruler. Meanwhile, the more complex parameters were the inertia J and the center of mass distance. Initially, these were approximated by considering the inertia of a prism characterized by a volume containing the drone. Subsequently, they were accurately calculated using a CAD model found on OneShape. In fact, upon finding the 3D model, the OneShape website provides the option to select a material for each component of the model. In our case, we considered PVC for the frame and wheel rims, rubber for the tire treads, aluminum and silicon for the microcontroller; the connecting wires for the LEDs and motors were removed from 3D model because they have negligible influence on the mass and inertia of the object.

Given that typically differential mobile robots are actuated by DC motors, which produce torques on the motors, we can introduce the actuated model that also includes the mathematical model of the motors. The mathematical model of a DC motor is as follows:

$$\left\{ \begin{array}{l} v_a = R_a i_a + L_a \frac{di_a}{dt} + e_a \\ e_a = K_b \omega_m \\ \tau_m = K_t i_a \\ \tau = N \tau_m \end{array} \right. \quad (2.5)$$

where i_a is the armature current, R_a and L_a are the resistance and inductance of the armature winding respectively, e_a is the back EMF, ω_m is the rotor angular speed, τ_m is the motor torque, K_t and K_b are the torque constant and back EMF constant respect. In the DDMR the motors are mechanically coupled to the robot wheels through the gears, the mechanical equations of motion of the motors are linked directly with the mechanical dynamics of the DDMR.

Therefore each dc motor will have

$$\begin{cases} \omega_{mR} = N\dot{\phi}_R \\ \omega_{mL} = N\dot{\phi}_L \end{cases}$$

In conclusion, the entire actuated model is described by this block diagram:

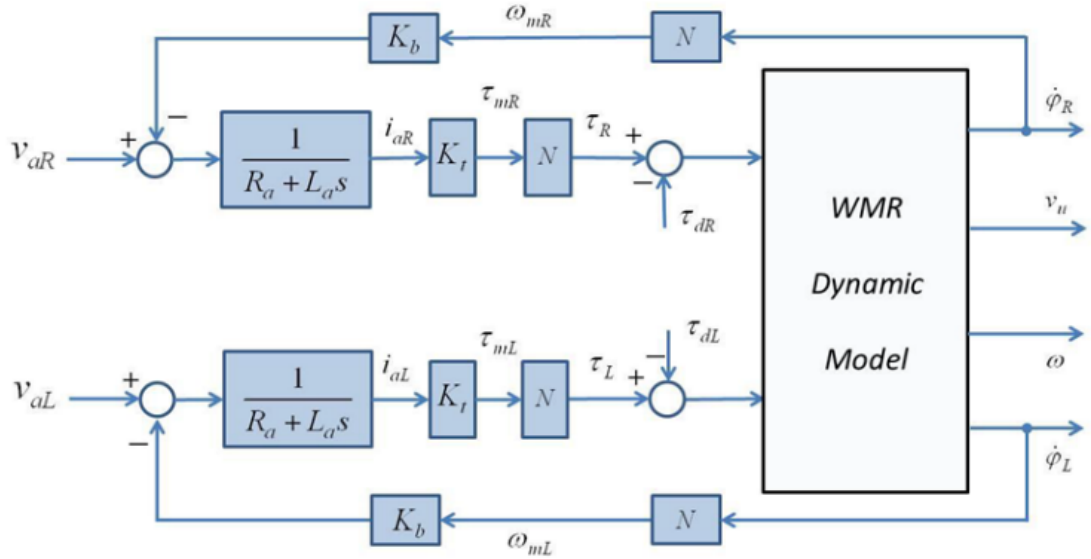


Figure 2.12: Dynamic model with actuators

Chapter 3

Ros toolbox

3.1 Integrating ROS Toolbox for Remote Communication in Duckiebot Control Design

To design a controller for the Duckiebot, overcoming challenges such as mapping pin configurations on the Raspberry board is crucial. While memory limitations were not a concern, the obscured mapping of pins presented a unique obstacle. In response, the ROS Toolbox in MATLAB emerged as a pivotal solution, seamlessly integrating MATLAB, Simulink, and the ROS network for ROS-centric applications. Simulink, complemented by the ROS Toolbox, provides a convenient and clean approach for effective communication with a remote ROS network.

Within the Duckiebot ROS network, custom messages, including those specific to Duckietown, were manually included and incorporated into the MATLAB path. After identifying the Duckiebot's IP address, a ROS master was established in MATLAB, enabling a global node on the ROS network. This setup facilitated subscription and publication on the Duckiebot's topic, with a primary focus on subscribing to encoder topics and publishing on motor drive topics to command velocity.

Publication was achieved using the ROS Toolbox's publisher block on the *wheels_driver_node/wheels_cmd* topic, allowing the observation of wheel motion on the Duckiebot.

The subsequent phase involved incorporating a subscriber block in Simulink to read messages about the angular position of wheels. Subscription to the *right(left)_wheel_encoder_node/tick* topic was necessary.

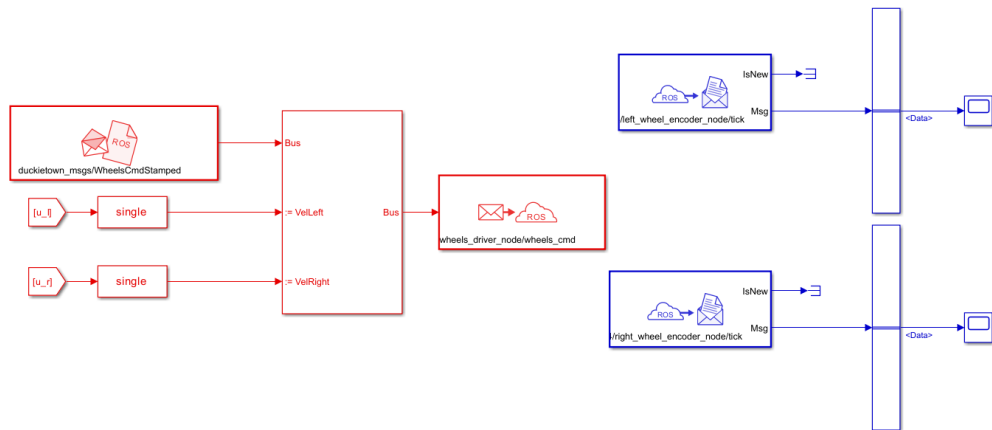


Figure 3.1: Simulink scheme for publishing and subscribing

Similarly, to achieve the image from the camera on the car, it was

necessary subscribing to the camera topic (`/camera_node/image/compressed`) and displaying the detected image on screen was achieved using a Video Viewer block, contingent on the installation of the Computer Vision Toolbox in MATLAB. Simulink, empowered by the ROS Toolbox, offers a convenient and efficient means of overcoming pin configuration challenges and communicating with a remote ROS network.

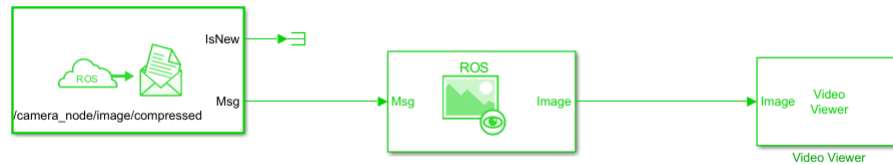


Figure 3.2: Simulink scheme for the camera

3.1.1 Modelling of the ROS command

In order to tune the PID able to adapte the real velocity to the desired one we need to model the ROS command useful to force the motors.

Basically we have modelled it as the DDMR model, which is torque controlled, controlled with a velocity feedback and a PID for each velocity of the wheel tuned on a sinusoidal signal.

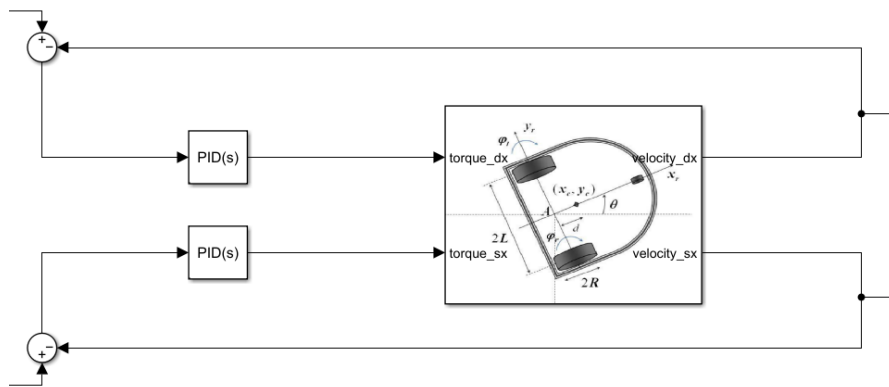


Figure 3.3: Modelling of the ROS command

Chapter 4

Inner Loop

4.1 Reconstruction of the velocity from the encoder data

To control the differential drive robot, a velocity control loop is designed, as ROS topics restrict the provision of torques, allowing only velocity values. Retrieving velocity directly from ROS topics, particularly as angular position, is unfeasible. Consequently, numerical reconstruction of velocity becomes essential. Initialization involves a ROS block receiving messages containing tick values detected by the wheel encoder. Utilizing a bus selector extracts the tick values from the message's `Data` field. Conversion of these values to a double variable is necessary for subsequent computations. The computation involves $x(t) - x(t - 1)$, representing the tick values at times t and $t - 1$. A delay block introduces a time delay to the signal,

followed by a first-order hold filter ensuring signal constancy for a given time instant. Conversion of tick information to rpm and subsequently to m/s is achieved using a combination of gains. Finally, a low-pass filter with a pole at 0.5 filters out noise affecting the signals. In the velocity reconstruction process, a timed switch with a duration of 13 seconds has been introduced. This prevents the transient effects of the reconstructive filter from impacting the actuators, potentially causing damage. After these 13 seconds, the loop is effectively closed, and after a brief transient period, the wheels reach the desired regime.

4.2 Velocity Control and Dynamic Simulation

In Chapter 3, we discussed the selection of a desired velocity and its transmission as a message to the ROS command for motor control. However, empirical testing revealed a significant discrepancy between the actual and desired velocities. To address this, a controller is needed to minimize the error computed from the encoder data. Additionally, the feedforward action based on the ROS command results in the two wheels moving at different velocities.

To narrow the gap between the reference and actual velocities, we propose designing a PID controller. The controller is modeled

on the DDMR (Duckiebot Dynamic Motion Model), incorporating torque control and velocity feedback. Subsequently, PID controllers are tuned individually for each wheel velocity based on a sinusoidal signal.

4.3 Dynamic Modeling and Control Adaptation

In accurately simulating the dynamic and kinematic behavior of the Duckiebot, modeling not only the dynamic system but also the pre-existing controller within the ROS network is crucial. Utilizing the ROS toolbox allows the direct transmission of velocity commands to the wheels. However, controlling the torque directly is not feasible, necessitating adaptation. There exists a pre-installed Raspberry Pi controller that actuates the wheels using the reference command from MATLAB. However, this control proved ineffective, with a negligible velocity error only when the wheels were spun freely. On the ground, they were slower, indicating that friction was not considered in the initial modeling.

To calibrate the PIDs effectively, identical for each wheel, it was necessary to model this phantom controller on the dynamic system. This ensures that the resulting scheme accurately represents the kinematic model, viewed as a black box, to which we directly

provide the reference. The internal controller is conceived as a low-performance PID, simulating the friction problem. Subsequently, a functional PID is implemented on the rotational velocity of the kinematic system. Furthermore, it was added a simple logical switching controller to stop the wheels for emergencies, allowing us to manually impose null velocities to the ROS publisher if something went wrong.

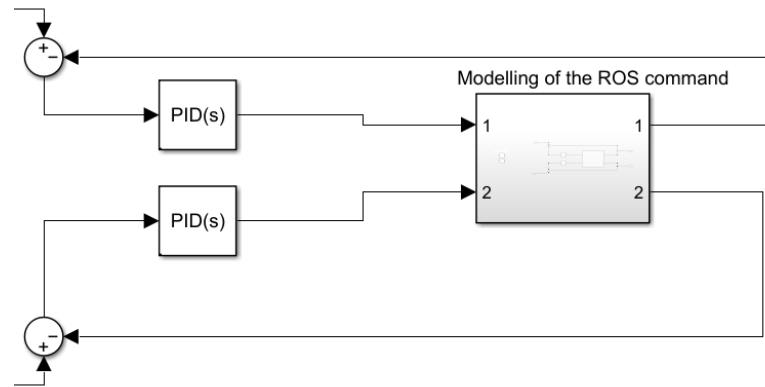


Figure 4.1: Scheme for the PID tuning

In order to protect the motors we inserted a saturation between 0.95 and -0.95 . The velocity feedback is the following:

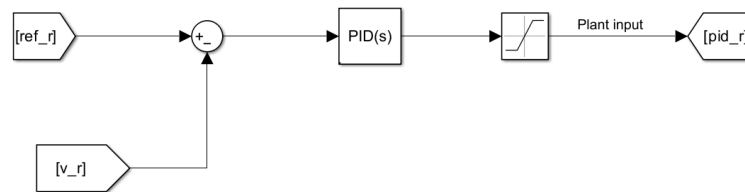


Figure 4.2: Scheme for the velocity feedback

Chapter 5

Outer Loop

5.1 Colour Detection

As we have seen in the chapter 3 we are able to communicate with the camera through ROS commands. The key idea of the first high level control is to arrest the Duckiebot in a small interval of time after detecting an object of a specific colour, in particular the red one.

Firstly we catch the image in order to split it into three matrix for the Red, Green and Blue part of the image, and then we wrote a MATLAB function, able to detect a red object in the center of the image, which gives a boolean output that has a *true* value iff a red object has been detected. To do that we need to impose a treshold and a condition which means that there is a red object in front of the camera: when the two conditions are fulfilled the boolean output

becomes *true* and then the Duckiebot gets stuck.

A logical switch takes in input the desired control action, able to make the error on the desired velocity as small as possible, and the boolean of the colour detection. As soon as the block detects the desired colour the control input to the ROS command is switched to zero.

Notice that when the boolean of the colour detection becomes false back, the desired velocity returns to be real control input so the Duckiebot restarts its motion.

5.2 Line Detection

The line detection constitutes the foundational concept underlying lane keeping in the context of autonomous driving. It involves identifying lines in a live video sourced from a camera using the Hough transform. The Hough transform is a method employed in image processing and computer vision to detect shapes, particularly lines or geometric forms such as circles or straight segments. Instead of directly identifying lines within the image, the Hough transform converts image points into a parametric space (referred to as the Hough space) where lines are represented by points. This method enables the identification of lines even if they are partially obscured or frag-

mented in the original image. The video stream originates from the ROS Subscriber, which retrieves the 'compressedImage' message and converts it into an RGB-formatted image using the 'Read Image' block. The resulting image manifests as a three-dimensional matrix with dimensions of 480x640x3. The initial two values denote the length and width of the video frame, respectively, while the third signifies three color levels composing the image. This image is subsequently converted into grayscale and cropped to obtain the lower half of the frame, ensuring that the lines can be exclusively identified on the road surface.

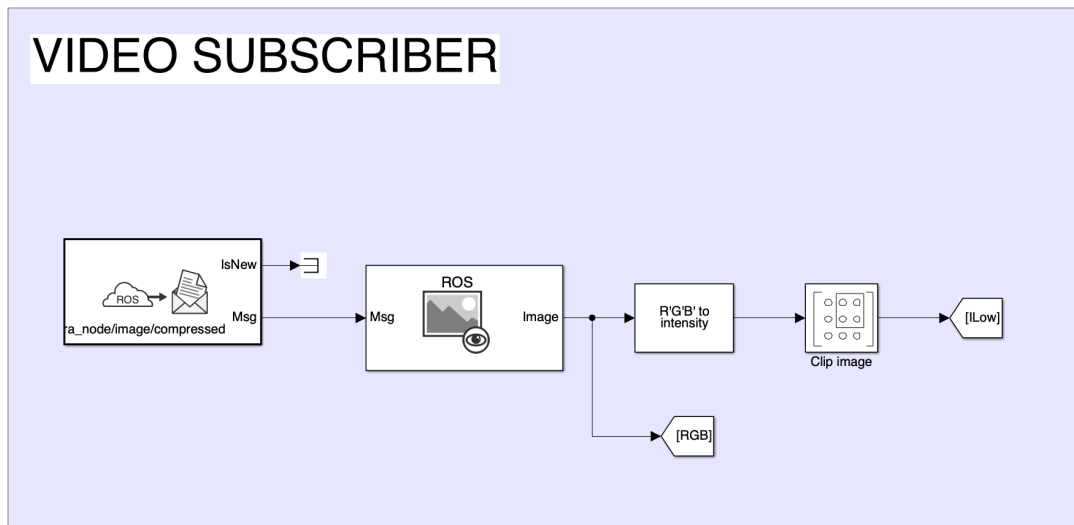


Figure 5.1: Camera ROS Subscriber

Subsequently, the image undergoes filtering, and through an Image Thresholding block, an image in black and white is obtained based on the grayscale intensity of each pixel. The image is now prepared to be processed by the Hough transform to identify the

lane lines.

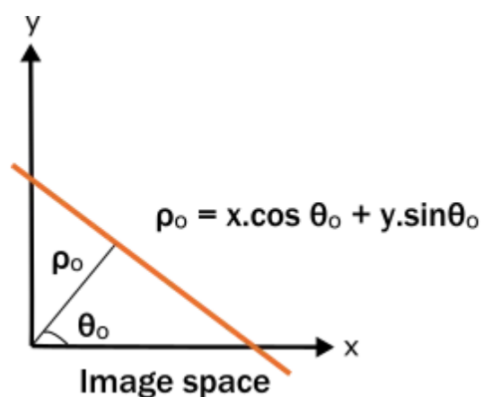


Figure 5.2: Hough Trasform

Then we utilize the Hough Transform block to acquire the Hough point space and the associated rho and theta values corresponding to the identified lines. Subsequently, the Find Local Maxima block is employed to locate the endpoints of the two major lines representing the extremities of the segments for the two lines. Utilizing a selector, we extract the indices that enable us to identify the rho and theta values linked to the lane lines.

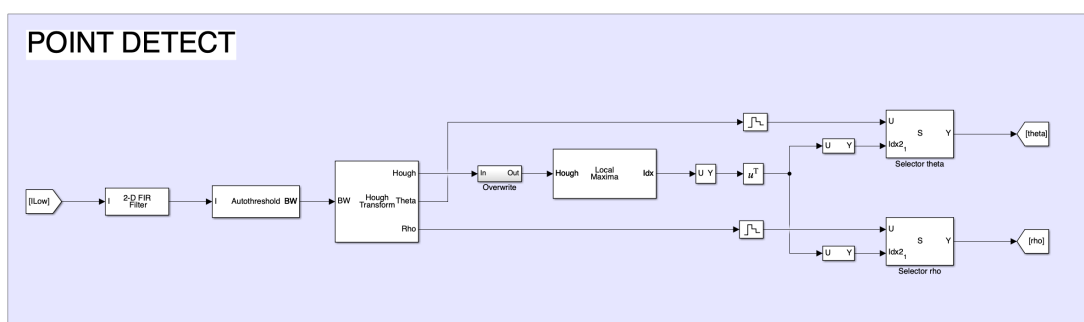


Figure 5.3: Point Detect

5.3 Line Keeping

Lane keeping, a concept primarily applied in autonomous driving and Advanced Driver Assistance Systems (ADAS), is designed to help vehicles maintain a central position within their lane. This enhances safety and reduces the risk of collisions due to accidental lane departures. In this chapter, we will examine our lane keeping system, focusing on the use of the camera on the Duckiebot. The process begins with lane detection algorithms identifying the road markings. Once the lane lines are detected, the system calculates a central line. The error is then calculated as the difference between the Duckiebot's current position and the central line position. Starting from the theta and rho values obtained from the line detection, we utilize the Hough Lines block to compute the positional values of the endpoints of the lines in terms of pixels within the video frame matrix. The road markings are represented by pairs of points, each defined by two coordinates that indicate its position within the pixel matrix. The central line, representing the desired trajectory, is identified by calculating the midpoints. Specifically, we calculate the midpoints between Y_a and Y_c , and between X_a and X_c , to determine the coordinates of the segment's upper point. Similarly, midpoints between Y_c and Y_d , and between X_b and X_d , are calculated for the lower point coordinates.

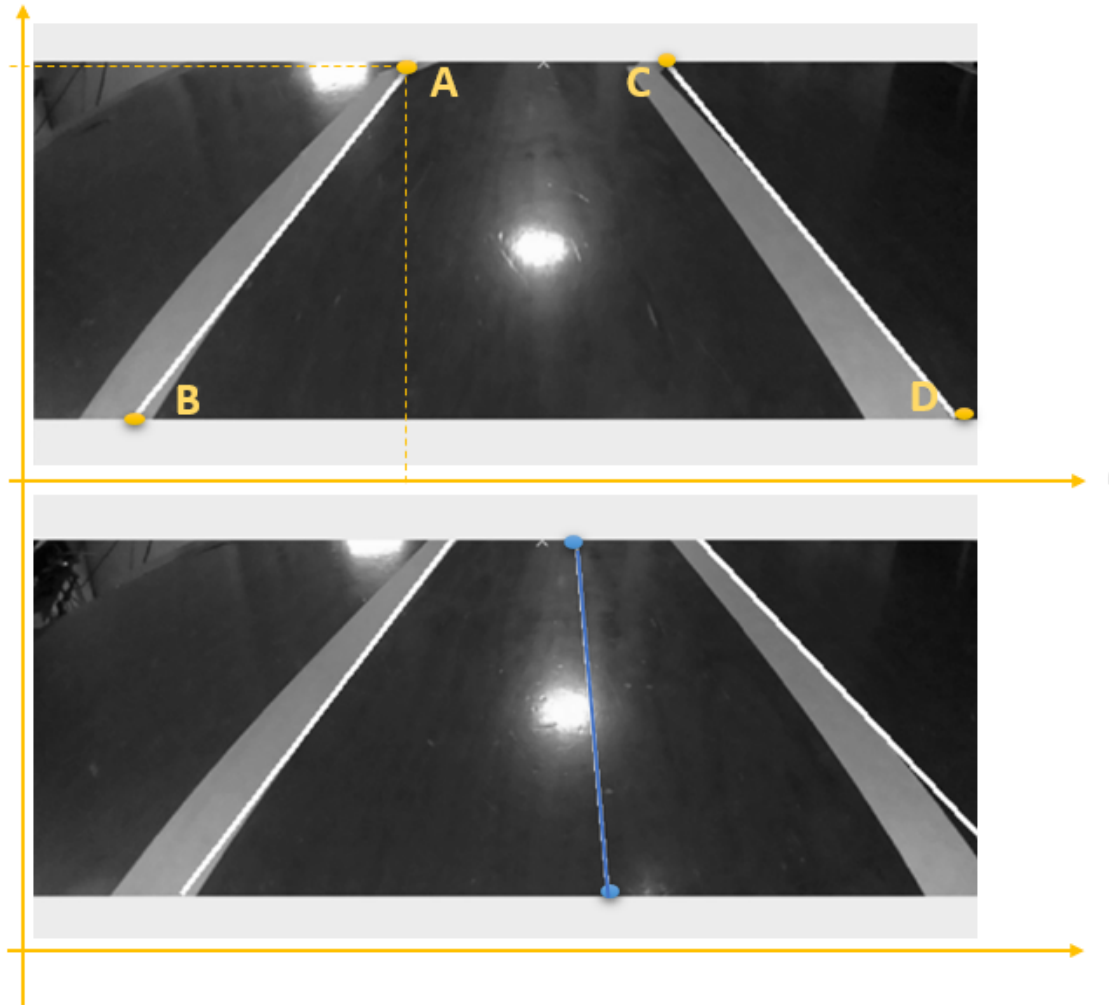


Figure 5.4: Camera Lines

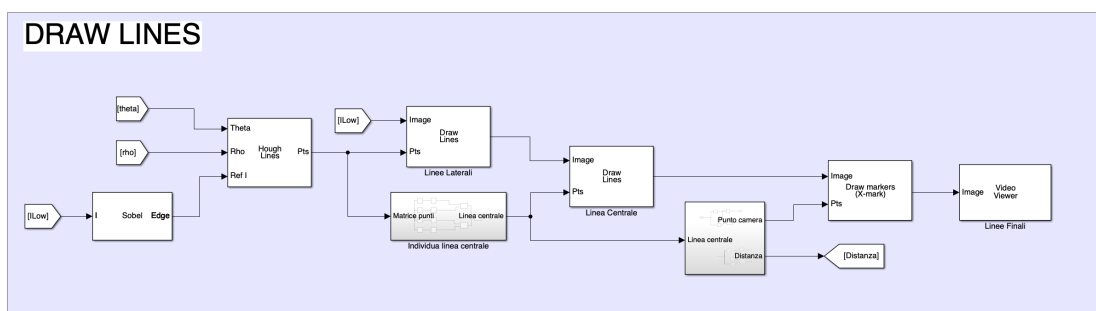


Figure 5.5: Draw Lines

The obtained values are concatenated into two two-element vectors:

$[Y_{mid_upper}, X_{mid_upper}]$ and $[Y_{mid_lower}, X_{mid_lower}]$. Then, another con-

catenate block combines these vectors into a single vector $[Y_{mid_upper}, X_{mid_upper}, Y_{mid_lower}, X_{mid_lower}]$. This data structure is necessary for the DrawLine function, which displays the line on the video. The error used for feedback is calculated by comparing the X_{upper} value with the central value of the entire camera's view. This central value exactly matches half the width of the camera's pixel matrix, representing the central point of the entire field of vision.

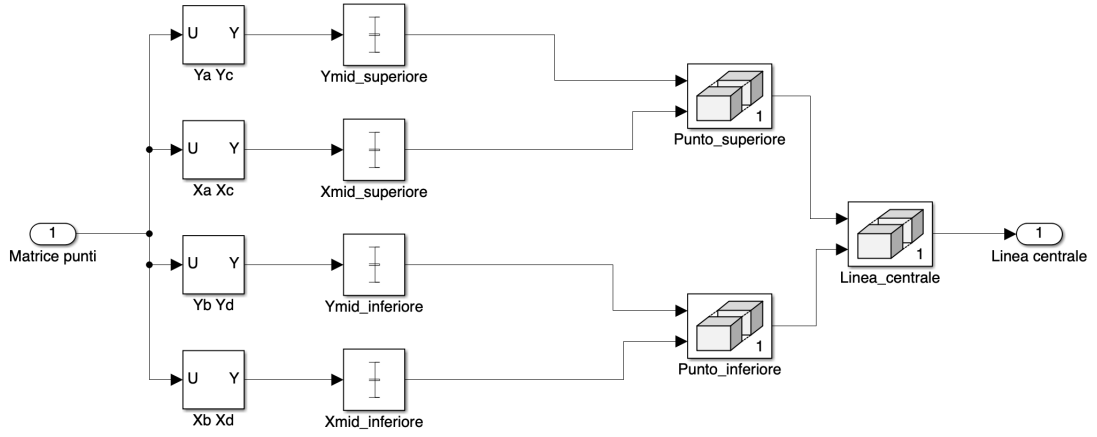


Figure 5.6: Point Matrix

The calculated error then becomes the input for the control system. Initially, an Switching control was implemented using Simulink's 'Chart' block, which is based on graphical programming and allows creating state diagram and finite state machine-based systems. The implemented logic is simple: if the error is greater than zero, the machine should rotate at a fixed speed to the left; if less than zero,

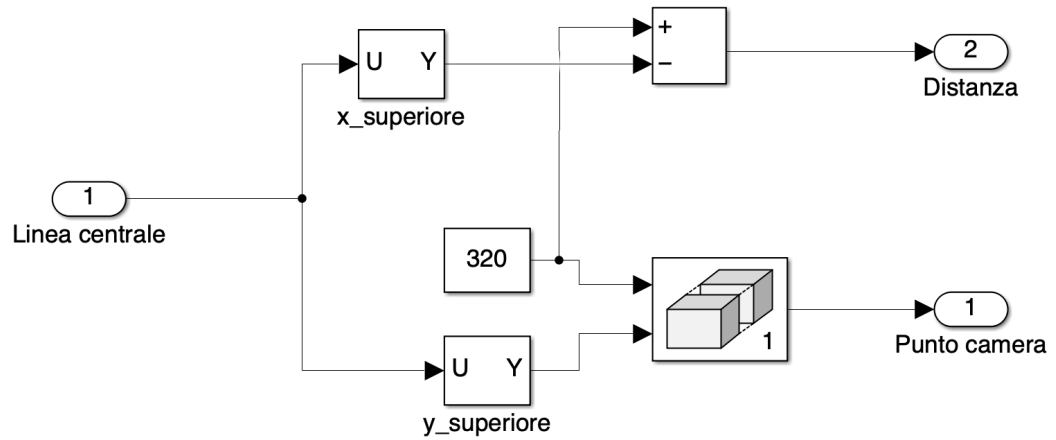


Figure 5.7: Central Line

to the right, while maintaining a constant tangential speed. To avoid constant state changes, an epsilon value was defined to establish a third state, "StraightOn", where the machine proceeds with zero rotation speed.

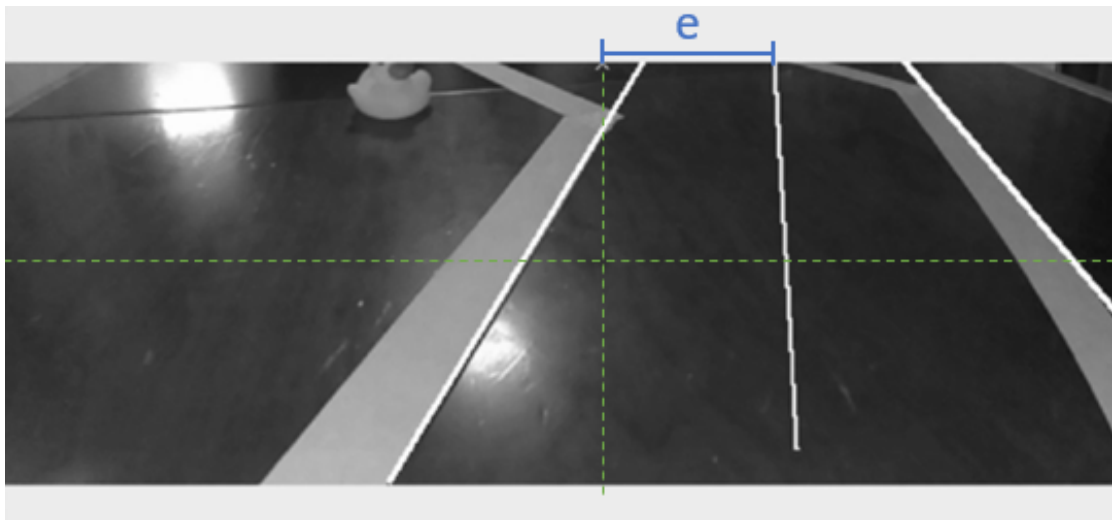


Figure 5.8: Error

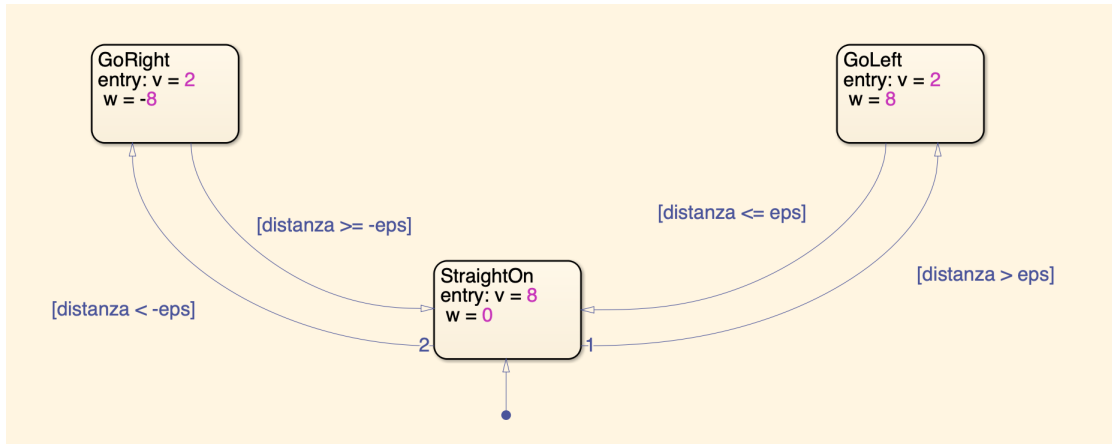


Figure 5.9: Flow Chart

For more accurate control, a PID control was later implemented. Based on the error, it generates the rotation speed to command the machine, always maintaining a constant tangential speed. The PID values, lacking an accurate model, were chosen based on various experiments. From the tangential and rotational speeds, the desired wheel rotation speeds were calculated using the inverse kinematics model.

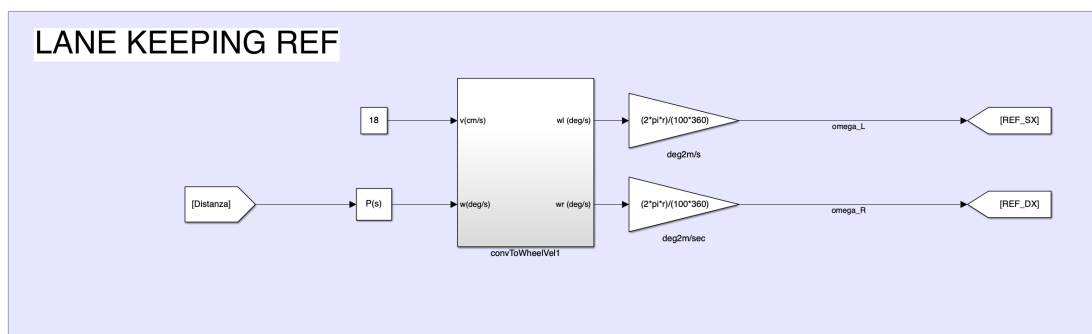


Figure 5.10: Lane Keeping Ref

5.4 Control Merge

After separately developing the feedback control models for speed and lane-keeping, we attempted to merge both controls to achieve a more precise and accurate control system. We combined the controls by using the wheel speed reference obtained from lane-keeping as feedback with the actual wheel speed reconstructed from the encoder outputs. However, we encountered a significant slowdown in Simulink's simulation time due to the ROS camera subscriber within the lane-keeping model. This introduced substantial latency, hindering the proper functioning of the speed control scheme. Upon considering that directly loading the Simulink model onto the DuckieBot might resolve latency issues, we faced an additional error while attempting to compile the code using the ROS toolbox. This error stemmed from the ROS architecture on which DuckieBot is based, employing Docker technology.

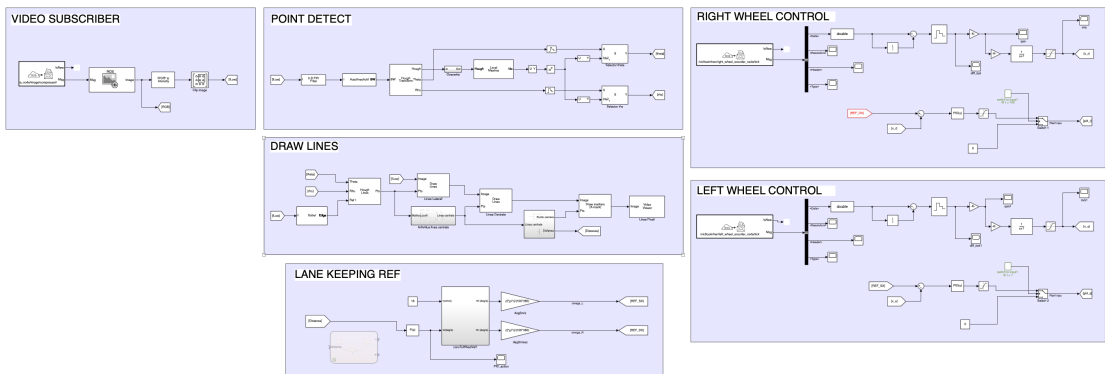


Figure 5.11: Cruise control and lane keeping

Chapter 6

Conclusion

In conclusion, the Duckietown project is a wellspring of possibilities and free will, notwithstanding the evident preference for ROS—a legitimate choice for a mobile robot. From the basic actuation of motors to encoder readings and up to image processing, it has been a continuous evolution of knowledge and discoveries that has certainly broadened our perspective, clearing a path beyond the clouds of commonplace simulations. Despite being at times frustrating, the journey has been rich in stimuli and captivating. We embarked on this endeavor with little to no prior knowledge in the field, aside from control, and emerged significantly enriched. The array of functions and control schemes applicable to the Duckiebot is astonishing, not to mention the possibilities associated with the camera and the extensive range of applications involving image processing. The path was winding, challenging, and uphill, but the sense of accomplish-

ment was ever-present. If you find yourself working with a Duckiebot, do not confine yourself to what you already know—evolving toward new techniques and knowledge is imperative!